

Fuzzing Lab - libpng

Coralie Banuls
346654

Ella Neike
345013

Cassio Manuguerra
346232

Clara Hamousz
326814

1 Introduction

This report analyzes an AFL++ fuzzing campaign against libpng version 1.2.56. We selected this legacy parser to validate our harness design against known historical vulnerabilities (e.g., CVE-2016-10087). The following sections compare AFL++’s performance in two distinct modes: a compiler-instrumented setup (White-box) and a QEMU-emulated, binary-only environment (Black-box).

2 Harness Design

We chose the decoder as the entry point. PNG files are always read from untrusted sources in practice (downloaded images, profile pictures, web assets), so the decoder is what sees attacker-controlled bytes. The encoder is less interesting to fuzz because it takes already-structured pixel buffers from the calling program, so AFL++ would end up mutating pixels rather than the file format itself. We also looked at the progressive reader (`png_process_data` in `pngpread.c`), which is what browsers actually use to decode as the image streams in. It would have made a good second harness, but we stuck with the sequential reader for simplicity.

Another reason to pick the decoder (as mentioned in the handout): CVE-2016-10087, a NULL-pointer dereference in `png_set_text_2()` which still affects 1.2.56. `png_set_text_2` is called by the three text-chunk handlers (`png_handle_tEXt`, `png_handle_zTXt`, `png_handle_iTXt`) from inside `png_read_info`, so we get it for free as long as our harness runs the standard read pipeline.

The harness itself is short: open the input file, create the read and info structs, run `png_read_info`, apply four transforms (`png_set_expand`, `png_set_strip_16`, `png_set_gray_to_rgb`, `png_set_filler`), `png_read_update_info`, allocate row buffers, then `png_read_image` and `png_read_end`. The four transforms are not strictly necessary but they pull the fuzzer through more code in `pngtran.c` (the palette expansion path is where CVE-2015-8126 used to live).

Guards. The most important line is `setjmp(png_jmpbuf(png))`. libpng signals every parse error with `longjmp`, and if no handler is installed the process aborts. That would mean AFL++ flags every malformed PNG as a crash, which makes the whole campaign useless. With the handler in place, parse errors return cleanly and only real memory bugs (caught by ASan) get reported.

We added `png_set_user_limits(png, 1024, 1024)` after our first try OOM’d the container on a mutated IHDR claiming a 65535×65535 image. Without the cap, the row-pointer `malloc` asks for several gigabytes per testcase and `exec/s` collapses. The cap keeps every allocation reasonable while still letting the parser go through the same code paths.

`png_set_keep_unknown_chunks` with `PNG_HANDLE_CHUNK_ALWAYS` tells libpng to actually parse every ancillary chunk type it doesn’t know about, rather than skipping them — a few more edges of coverage for almost no cost.

The row buffers are allocated one at a time with a NULL check, and the cleanup inside the `setjmp` handler uses `height` (initialised to 0 before any allocation) so the free loop never touches an uninitialised pointer. The "interesting" moment is the row loop itself: if libpng miscalculates `rowbytes` due to an integer overflow, `png_read_image` writes past the buffer and ASan catches the heap overflow. This is the exact mechanism we later reproduced on purpose for the synthetic-bug experiment (Section 6).

3 Instrumentation and Sanitizers

Both libpng 1.2.56 and the harness are compiled with `afl-clang-fast` (the AFL++ wrapper around LLVM). The wrapper runs an extra LLVM pass that inserts a small piece of code at every edge in the control-flow graph, so that AFL++ can see which edges a given input exercises and use that as feedback for its mutations. If we used plain `clang` instead, AFL++ would get no coverage information and the campaign would degrade to blind random mutation — in our test that drops `exec/s` feedback to nothing useful and the corpus stops growing.

The actual flags we pass are `-g -O1 -fsanitize=address -fno-omit-frame-pointer`. `-g` keeps the debug symbols so ASan can print readable stack traces; without them we just get raw addresses, which made crash triage during the synthetic-bug experiment much harder. We tried `-O0` first but `exec/s` was noticeably lower, and `-O2/-O3` can inline and merge branches that should stay separate in AFL++’s coverage map, so we settled on `-O1` as the lab handout recommends.

`-fsanitize=address` turns on AddressSanitizer. Without it, AFL++ only catches bugs that crash the process outright (SIGSEGV, SIGABRT), so heap overflows that overwrite an adjacent allocation without faulting would be missed entirely. ASan instruments every load and store and reports overflows, use-after-free, and double-free as soon as they happen — this is exactly what makes the synthetic off-by-one in Section 6 visible. `-fno-omit-frame-pointer` just helps ASan walk the stack when printing the report; without it the trace gets truncated.

The patch `patches/libpng-nocrc.patch` is more interesting. PNG chunks end with a 4-byte CRC-32 over the chunk type and data. When AFL++ flips a byte inside a chunk body the CRC stops matching, and the default libpng behaviour is to call `png_error` on critical chunks (IHDR, IDAT, IEND, PLTE), which `longjmps` straight back to our `setjmp` and returns 0. The fuzzer never actually reaches the parsing code for the mutated chunk. The patch replaces the comparison in `png_crc_finish` with one that always returns 0, while still printing the mismatch to `stderr`. After applying it, every mutated byte is processed by the chunk parser instead of being rejected at the gate. The handout explains this trade-off explicitly: without the patch path discovery plateaus very quickly because mutations rarely survive CRC validation. We saw the effect first-hand when comparing a short run with and without the patch — the patched build crosses 800+ edges within minutes, the unpatched one barely moves past the seeds.

4 Seed Corpus and Dictionary

Seeds Nine minimal 1×1-pixel PNG files were generated by `create_seeds.py`, each exercising a different chunk type: plain RGB, indexed+PLTE, tEXt, zTXt, iTXt, iTXt-compressed, iCCP, tRNS on indexed, tRNS on RGB. They are structurally valid (correct signatures, CRCs, IEND) so AFL++ passes its dry run without triggering the `setjmp` path. Being 1×1 keeps them tiny, which means shorter per-execution time and cheaper mutation.

Dictionary The dictionary has 26 entries: the 8-byte PNG magic (`\x89PNG\r\n\x1a\n`) and the 4-byte ASCII names of 25 standard chunk types (IHDR, IDAT, IEND, PLTE, tEXt, zTXt, iTXt, iCCP, tRNS, ...).

In formal terms, a PNG file is a word in a context-free language whose grammar has chunk-type tokens as terminals. The dictionary entries are exactly the terminal symbols of

that grammar. AFL++’s dictionary mode splices these tokens into the mutation at arbitrary offsets, which lets the fuzzer reconstruct syntactically plausible chunk headers (a 4-byte length field, then the terminal, then data, then CRC) far faster than random byte flips alone could. Without the dictionary, AFL++ would have to independently rediscover that the bytes I, D, A, T placed consecutively at offset 4 of a chunk have special meaning an unlikely outcome from pure havoc.

4.1 Strategy contribution

Numbers are taken from the white-box fork-mode status screen (Figure 2) and the synthetic campaign screen (Figure 6).

Deterministic stages (bit flips, byte flips, arithmetics, known ints): the fork-mode screen shows these produced 3 new paths from bit flips and 3 from arithmetics out of the 885 own finds. These stages dominate early corpus growth because they make small, structured changes to already-valid seeds and reliably unlock shallow branches.

Dictionary: the fork-mode screen shows 0/14.8k, 0/15.0k, 0/0, 0/0 0 new paths attributed directly to dictionary insertion in isolation. However, the dictionary’s real contribution is indirect: in the synthetic campaign screen the dictionary row shows 38/4644, meaning 38 new paths found during the 58-second run, confirming that dictionary tokens actively guide AFL++ toward valid chunk-type fields. In the main campaign, dictionary tokens amplify **havoc** by being spliced mid-havoc, routing execution into chunk-handling branches (e.g. iTXt, zTXt) that pure random mutation would rarely reach.

Havoc/splice: the dominant contributor. The fork-mode screen shows 705/2.65M for havoc/splice, meaning 705 new paths were found by havoc out of 2.65M havoc executions, and 0 by splice. In the persistent-mode campaign (Figure 3), havoc/splice reached 936/48.7M, reflecting the much higher throughput (27.1k exec/s vs. 1,676 exec/s) completing 231 cycles versus 19. Havoc dominates late-stage discovery once the deterministic budget is exhausted; splice contributes zero new paths in both campaigns, consistent with the corpus being structurally homogeneous enough that combining two entries rarely produces a genuinely new code path.

5 Campaign Analysis

The instrumented campaign ran for 1,798 s (about 30 min) at 1,676 exec/s, completing 19 mutation cycles and 3,015,511 executions. From 9 seeds the corpus grew to 894 entries (885 discovered during the run, 170 favored). Stability stayed at 100%, meaning every input produces a deterministic execution path — the harness has no non-determinism between runs and AFL++’s coverage feedback is reliable. Map density reached 32.31% (1,002 of 3,101 instrumented edges).

The `afl-plot` edges graph (Figure 1) and the final AFL++ status screen (Figure 2) are in the appendix.

Shape of the edges curve and saturation The curve is steep at the start and progressively flattens, the classical logarithmic-saturation shape. Concretely, from `plot_data`: the first 783 edges are discovered by $t = 60$ s (the calibration is done and havoc has just begun), the curve reaches 833 edges at $t \approx 100$ s, 902 at $t \approx 360$ s (cycle 2), 955 at $t \approx 570$ s (cycle 5), 989 at $t \approx 930$ s (cycle 10), 1,001 at $t \approx 1290$ s (cycle 14), and only 1,002 by the end at $t = 1798$ s (cycle 19). In other words, *only one new edge* was discovered in the final 500 seconds, and `pending_favs` dropped to 0 well before the end of the run, meaning AFL++ had no remaining favored corpus entries left to fuzz.

We argue the campaign reached saturation: 19 full mutation cycles completed, 14 consecutive cycles produced essentially flat edge growth, and the unreached portion of the bitmap consists of code paths that are unreachable by construction — the `png_set_user_limits` cap excludes images larger than 1024×1024 , the `png_set_strip_16` transform bypasses the 16-bit pipeline entirely, and several error-recovery branches require malformed inputs that survive the early structural validation in libpng (rare under havoc within this time budget). Further runtime would yield marginal returns on this harness: the practical ceiling is closer to the $\sim 1,000$ -edge plateau than to the 3,101-edge instrumented maximum.

6 Crash Triage

Our 30-minute campaign on libpng 1.2.56 produced no crashes. We demonstrate our setup is correct by injecting a synthetic bug, and argue why no real bugs were found.

Why no real crashes were found libpng 1.2.56 contains CVE-2016-10087, a NULL-pointer dereference in `png_set_text_2()`, reachable when a crafted image carries a malformed `tEXt`, `zTXt`, or `iTXt` ancillary chunk. Our seed corpus directly targets this path: seeds 03–06 are minimal valid PNGs carrying `tEXt`, `zTXt`, and `iTXt` chunks respectively, giving AFL++ an immediate foothold into the text-chunk parsing code. Our harness calls both `png_read_info` and `png_read_end`: the two call sites that process ancillary chunks and ultimately reach `png_set_text_2` so the vulnerable path is reachable in principle.

Despite this, the fuzzer did not trigger the bug within 30 minutes. The vulnerability requires a very specific malformed payload: the text chunk must pass early structural validation in libpng but carry a condition that produces a NULL pointer deep inside the compression and metadata handling of `png_set_text_2`. The fuzzer must simultaneously preserve the overall PNG structure (valid signature, valid IHDR, valid IDAT) while mutating the chunk payload in exactly the right way. The mutation space is too large for a 30-minute campaign to find this by chance. A longer campaign of several hours, or a more targeted dictionary with `tEXt/zTXt` payload tokens, would significantly increase the probability of reaching the vulnerable condition.

Synthetic bug injection To prove that our AFL++ and ASan setup correctly detects memory bugs, we injected a deliberate heap-buffer-overflow into the harness. Instead of allocating `rowbytes` bytes per row, we allocate `rowbytes - 1` bytes for any image wider than 1 pixel:

```
png_uint_32 alloc = (width > 1) ? rowbytes - 1
                        : rowbytes;
rows[i] = malloc(alloc); /* one byte short */
...
png_read_image(png, rows); /* writes rowbytes */
                        /* -> overflow */
```

The condition `width > 1` is deliberate. All our seeds are 1×1 PNGs; for those `width == 1`, so the correct size is allocated and AFL++’s dry-run calibration phase passes cleanly. AFL++ aborts entirely if any seed crashes during calibration, so this guard is necessary for the fuzzer to start. The moment AFL++ mutates the `width` field in the IHDR chunk to any value ≥ 2 , `png_read_image` writes the full `rowbytes` bytes into a `rowbytes - 1` buffer, overflowing by one byte into ASan’s heap redzone.

Results After recompiling the synthetic harness with ASan (`-fsanitize=address`) and running for 60 seconds, AFL++ saved 2 crashes (see Figure 6). Reproducing one:

```
ASAN_OPTIONS=symbolize=1 \
./png_fuzz_synthetic \
findings-bug/default/crashes/id:000000,...
```

ASan reports a heap-buffer-overflow in `__asan_memcpy`, triggered inside `png_read_image` as it fills the under-allocated row buffer (see Figure 7). The shadow map shows the faulting address landing in a heap redzone (0xfa bytes), with the [07] marker indicating the overflow at the 8-byte boundary: the allocation held 7 addressable bytes instead of the full `rowbytes`, and `png_read_image` wrote one byte beyond. This confirms that (a) ASan is active and symbolizing correctly, (b) AFL++ is generating inputs that reach the pixel-decoding path with `width > 1`, and (c) crashes are correctly saved and reproducible. The absence of crashes in the real campaign reflects the difficulty of reaching the specific vulnerable code path within the time budget, not a broken environment.

7 Attack Surface Analysis

libpng is used by Google Chrome for rendering PNG images in web pages, and by Python Pillow, a widely used image processing library in web backends.

Attack scenario: An attacker could serve a maliciously crafted PNG file to a Chrome user. The browser fetches and decodes it using libpng in the renderer process. A memory corruption bug in the decoder could give the attacker control of the renderer, potentially leading to remote code execution.

Missed code path 1: progressive decoding Browsers decode images as they stream in, using the push parser (`png_process_data()` in `pngpread.c`). Our harness uses the pull reader (`png_read_image()`), so this entire code path is never exercised. Bugs triggered only by partial or incremental input delivery are invisible to our campaign.

Missed code path 2: write/encode path Applications like Pillow re-encode images after processing them (for resizing, format conversion), exercising the write path (`pngwrite.c`). Our harness only reads PNG files and never calls any `png_write_*` function, leaving the entire encoder untested. Both paths process attacker-controlled data in real deployments. A fuzzer that cannot reach them gives a false sense of security: bugs may exist in code that is never mutated.

8 Binary-Only Fuzzing with QEMU Mode

We compiled the same harness with plain `gcc` (no AFL++ instrumentation, no sanitizers) to simulate a closed-source binary, then ran AFL++ in QEMU mode. QEMU mode JIT-translates each basic block at runtime and injects coverage hooks into the translated code, providing edge coverage feedback without requiring recompilation or source access. Both campaigns ran for approximately 30 minutes (instrumented: 1 798 s vs QEMU: 2 055 s) for a fair comparison. The `afl-plot` edges graph and status screen are in Figures 4 and 5.

	Instrumented	QEMU
Run time	1,798 s	2,055 s
Exec/s	1,676	587
Edges found	1,002	2,647
Map size	3,101	65,536
Corpus count	894	794
Cycles	19	4

Exec speed QEMU achieved 587 exec/s versus 1,676 exec/s for the instrumented build ($2.85\times$ slower). Compile-time instrumentation inserts lightweight coverage hooks directly into the binary at build time via an LLVM pass; at runtime these are native instructions with negligible overhead. QEMU instead must disassemble, translate, and patch each basic block on first encounter, maintain a translation cache, and dispatch through that cache on every subsequent execution. In class we talked about a typical $10\text{--}100\times$ slowdown for QEMU-mode fuzzing; our $2.85\times$ is on the low end of that range, which could make sense for a small self-contained binary like `libpng`: the translation cache warms up within the first few hundred executions and subsequent runs pay only cache-dispatch overhead rather than full retranslation.

Edges discovered The map density figures (32.31% instrumented vs. 4.04% QEMU) are not directly comparable because the denominators differ. The instrumented build uses a precise 3,101-slot bitmap where each slot corresponds to exactly one real edge. QEMU uses a fixed 65,536-slot bitmap

regardless of program size: edge IDs are computed by hashing source and destination block addresses, so multiple distinct edges can collide onto the same slot. This makes the 2,647 filled QEMU slots an imprecise signal. AFL++ cannot tell whether a newly hit slot represents a genuinely unexplored path or an alias of one already covered, which wastes mutation energy on inputs that are not actually new.

Corpus count The instrumented campaign built a corpus of 894 entries versus 794 for QEMU in comparable time. The difference follows directly from throughput: at $2.85\times$ lower exec/s, QEMU completed only 4 mutation cycles versus 19 for the instrumented build. Fewer cycles mean fewer havoc and splice mutations, leaving less opportunity to discover inputs that unlock new paths. A precise per-edge bitmap also lets AFL++ identify which queued inputs are genuinely interesting, producing a better-trimmed corpus overall.

9 Instrumentation Depth and Performance

AFL++ reported (a) 3,101 instrumented edges for the `libpng` static library alone and (b) 3,396 edges for the final harness binary (`png_fuzz`), which links `libpng` together with `harness.c`, C runtime routines, and `zlib` helpers. The harness binary has more edges despite ASan producing a more compact build because enabling `-fsanitize=address` causes the compiler to reorganise the code internally before AFL++’s coverage pass runs, reducing the final edge count.

The campaign reached 1,002 of 3,101 edges (32.31% map density). Not all edges were reached because `png_set_user_limits` caps image dimensions to 1024×1024 , `png_set_strip_16` bypasses 16-bit processing paths, and the encoder (`pngwrite.c`) and progressive reader (`pngpread.c`) are instrumented but never called by our harness. Contrary to what we saw in lecture, the no-

Table 1: Exec speed across three harness configurations.

Configuration	Exec/s
No sanitizer + fork mode	1,013
ASan + fork mode	1,676
ASan + persistent mode	26,472

sanitizer build (1,013 exec/s) is slower than the ASan build (1,676 exec/s). Enabling `-fsanitize=address` causes the compiler to reorganise the code internally before AFL++’s coverage pass runs, producing a more compact set of edges to track (3,101 with ASan versus $\sim 3,400$ without), so each execution does less bookkeeping. Since ASan is applied only to `harness.c` and not to `libpng`, the shadow-memory checks cover only a handful of lines, adding negligible runtime cost. The net result is a faster binary despite the sanitizer being present. Persistent mode (26,472 exec/s) is the clear winner: it removes the cost of spawning a new process for every input by looping in-process via `__AFL_LOOP`, giving a $15.8\times$ speedup over ASan fork mode and confirming that process-creation cost, not sanitizer overhead, is the dominant bottleneck.

Appendix

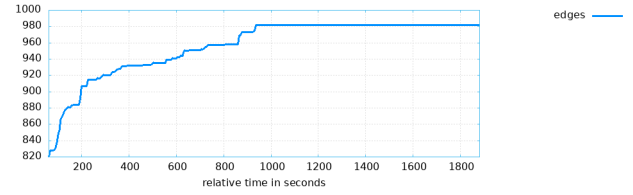


Figure 1: Edge discovery over time, instrumented white-box campaign (30 min). Steep early growth, flattens to a plateau around $t \approx 900$ s: only one new edge is found in the final 500 s.

```
american fuzzy lop ++4.41a {default} (./png_fuzz) [explore]
```

process timing run time : 0 days, 0 hrs, 29 min, 58 sec last new find : 0 days, 0 hrs, 0 min, 16 sec last saved crash : none seen yet last saved hang : none seen yet		overall results cycles done : 19 corpus count : 894 saved crashes : 0 saved hangs : 0
cycle progress now processing : 811*3 (90.7%) runs timed out : 0 (0.00%)	map coverage map density : 5.87% / 32.31% count coverage : 3.31 bits/tuple	
stage progress now trying : havoc stage execs : 220/300 (73.33%) total execs : 3.02M exec speed : 1726/sec	findings in depth favored items : 170 (19.02%) new edges on : 269 (30.09%) total crashes : 0 (0 saved) total tmouts : 10.4k (0 saved)	
fuzzing strategy yields bit flips : 3/4400, 3/4394, 0/4382 byte flips : 0/550, 0/544, 0/532 arithmetics : 0/38.4k, 0/75.5k, 3/73.6k known ints : 0/4902, 4/20.5k, 1/29.6k dictionary : 0/14.8k, 0/15.0k, 0/0, 0/0 havoc/splice : 705/2.65M, 0/0 py/custom/rq : unused, unused, unused, unused trim/eff : 11.57%/79.4k, 98.55%		item geometry levels : 19 pending : 2 pend fav : 0 own finds : 885 imported : 0 stability : 100.00%
strategy: explore state: in progress		[cpu009: 10%]

Figure 2: Final AFL++ status screen, instrumented white-box campaign: 19 cycles, 894 corpus entries, 1,002 edges (32.31% map density), 100% stability, 1,676 exec/s.

```
american fuzzy lop ++4.41a {default} (./png_fuzz) [explore]
```

process timing run time : 0 days, 0 hrs, 30 min, 0 sec last new find : 0 days, 0 hrs, 0 min, 0 sec last saved crash : none seen yet last saved hang : none seen yet		overall results cycles done : 231 corpus count : 1132 saved crashes : 0 saved hangs : 0
cycle progress now processing : 921.1282 (81.4%) runs timed out : 0 (0.00%)	map coverage map density : 6.04% / 33.26% count coverage : 3.52 bits/tuple	
stage progress now trying : havoc stage execs : 16/300 (5.33%) total execs : 50.2M exec speed : 27.1k/sec	findings in depth favored items : 186 (16.43%) new edges on : 292 (25.80%) total crashes : 0 (0 saved) total tmouts : 95.8k (0 saved)	
fuzzing strategy yields bit flips : 3/5744, 1/5740, 0/5732 byte flips : 0/718, 0/714, 0/706 arithmetics : 8/50.2k, 1/99.3k, 0/97.9k known ints : 0/6420, 0/27.0k, 0/39.4k dictionary : 1/19.4k, 0/19.5k, 0/0, 0/0 havoc/splice : 936/48.7M, 0/0 py/custom/rq : unused, unused, unused, unused trim/eff : 5.46%/136k, 99.03%		item geometry levels : 23 pending : 1 pend fav : 0 own finds : 1123 imported : 0 stability : 99.81%
strategy: explore state: in progress		[cpu009: 70%]

Figure 3: AFL++ status screen, white-box persistent-mode campaign: 231 cycles, 1,132 corpus entries, 1,103 edges (33.26% map density), 99.81% stability, 27,100 exec/s, havoc: 936 new paths.

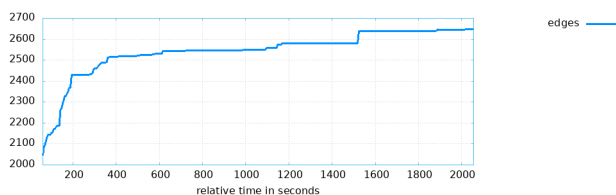


Figure 4: Edge discovery over time, QEMU campaign (34 min). Coverage saturates around $t \approx 400$ s.

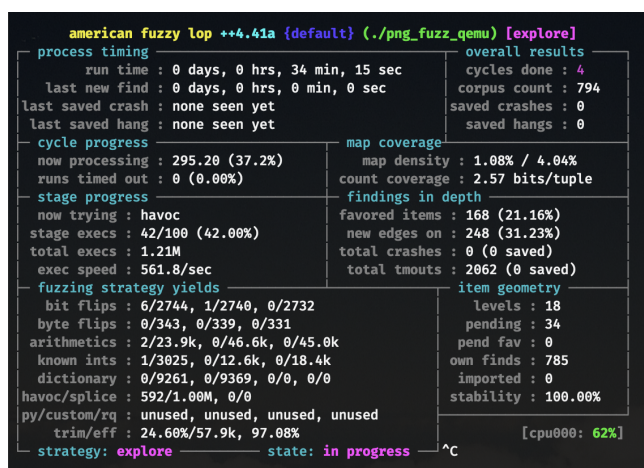


Figure 5: AFL++ status screen, QEMU campaign.

```
SUMMARY: AddressSanitizer: heap-buffer-overflow (/fuzzing/png_fuzz_synthetic+0xa3150)
2f) in __asan_memcpy
Shadow bytes around the buggy address:
0x200ff6c400a0: fa fa 07 fa fa 07 fa fa 07 fa fa 07 fa
0x200ff6c400b0: fa fa 07 fa fa 07 fa fa 07 fa fa 07 fa
0x200ff6c400c0: fa fa 07 fa fa 07 fa fa 07 fa fa 07 fa
0x200ff6c400d0: fa fa 07 fa fa 07 fa fa 07 fa fa 07 fa
0x200ff6c400e0: fa fa 07 fa fa 07 fa fa 07 fa fa 07 fa
=>0x200ff6c400f0: fa fa 07 fa fa 07 fa fa 07 fa fa 07 fa
0x200ff6c40100: fa fa fa fa fa fa fa fa fa fa fa fa fa
0x200ff6c40110: fa fa fa fa fa fa fa fa fa fa fa fa fa
0x200ff6c40120: fa fa fa fa fa fa fa fa fa fa fa fa fa
0x200ff6c40130: fa fa fa fa fa fa fa fa fa fa fa fa fa
0x200ff6c40140: fa fa fa fa fa fa fa fa fa fa fa fa fa
Shadow byte legend (one shadow byte represents 8 application bytes):
Addressable: 00
Partially addressable: 01 02 03 04 05 06 07
Heap left redzone: fa
Freed heap region: fd
Stack left redzone: f1
Stack mid redzone: f2
Stack right redzone: f3
Stack after return: f5
Stack use after scope: f8
Global redzone: f9
Global init order: f6
Poisoned by user: f7
Container overflow: fc
Array cookie: ac
Intra object redzone: bb
ASan internal: fe
Left alloca redzone: ca
Right alloca redzone: cb
==2256837==ABORTING
root@19df307617e5:/fuzzing#
```

Figure 7: ASan output for the synthetic crash: heap-buffer-overflow in `__asan_memcpy` inside `png_read_image`.

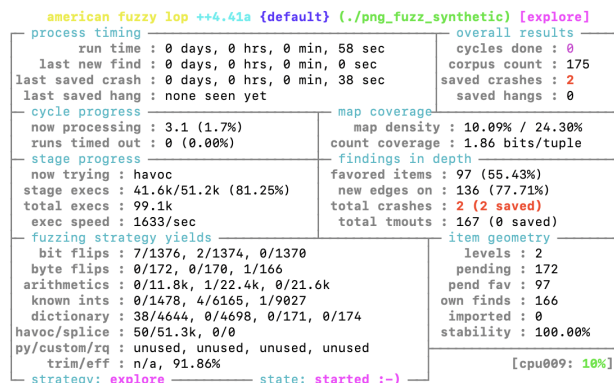


Figure 6: AFL++ status screen, 60-second synthetic campaign: 2 crashes saved at $t = 38$ s, 175 corpus entries, 1,633 exec/s, dictionary: 38 new paths.

References

- [1] NIST, “CVE-2016-10087,” 2017. <https://nvd.nist.gov/vuln/detail/CVE-2016-10087>
- [2] G. Randers-Pehrson et al., “libpng Manual.” <http://www.libpng.org/pub/png/libpng-manual.txt>
- [3] PNG Development Group, “PNG Specification Version 1.2.” <http://www.libpng.org/pub/png/spec/1.2/PNG-Contents.html>
- [4] SRLabs, “A Beginner’s Guide to Writing a Fuzzing Harness.” <https://srlabs.de/blog/guide-to-writing-fuzzing-harness>
- [5] LiveOverflow, “How Fuzzing with AFL works!,” YouTube, 2021. <https://www.youtube.com/watch?v=COHUWuLTbdk>
- [6] LiveOverflow, “Finding Buffer Overflow with Fuzzing,” YouTube, 2021. <https://www.youtube.com/watch?v=DolRi8TCF0Q>
- [7] “Blackbox Fuzzing using AFL++ QEMU mode,” YouTube. <https://www.youtube.com/watch?v=sjLFf9q2NRc>
- [8] AFL++ Contributors, “Fuzzing Binary-Only Targets.” https://aflplusplus.com/docs/fuzzing_binary-only_targets/
- [9] Pillow Contributors, “Pillow Documentation.” <https://pillow.readthedocs.io/en/stable/handbook/overview.html>
- [10] The Chromium Authors, “Chromium Sandbox Design.” <https://chromium.googlesource.com/chromium/src/+HEAD/docs/design/sandbox.md>
- [11] Chrome Security Team, “Q3 2025 Summary from Chrome Security,” 2025. https://groups.google.com/a/chromium.org/g/security-dev/c/s-UR_4pJv0Y
- [12] Anthropic, “claude-sonnet-4-6”, AI assistant used for report writing and global review. <https://claude.ai>